

Log In ▾

The wiki page is under active construction, expect bugs.

Metody prohledávání stavového prostoru, algoritmy. vector2 ▾

Reprezentace znalostí a rozhodování s nepřesnou znalostí.

Dvouhráčové hry.

B4B36ZUI  [Webové stránky předmětu](#) 

- **Známé systémy umělé inteligence** – DeepBlue, Watson, AlphaGo, Eliza, Shakey, DQN.
- **Formální reprezentace problému AI** – problém mnohorukého banditu, MDP, POMDP, hra v rozšířené formě.
- **Metody prohledávání stavového prostoru** – DFS, BFS, ID-DFS, Dijkstra, A*.
- **Algoritmy posilovaného učení** – policy evaluation, policy improvement, policy iteration, value iteration, Q-learning.
- **Algoritmy pro řešení her dvou hráčů** – minimax, alpha-beta prořezávání, negamax, niggascout, MCTS.
- **Strukturovaná reprezentace znalostí** – CSP, Scheduling, Situation calculus, STRIPS.
- **Neurčitost v AI** – maximalizace očekávané utility, Bayesovo pravidlo, Bayesovské sítě.
- **Řešení POMDP** – PBVI, HSVI.

1. Známé systémy umělé inteligence

DeepBlue, Watson, AlphaGo, Eliza, Shakey, DQN

DeepBlue

- **Vývojce:** IBM
- **Účel:** Hraní šachů.
- **Popis:** První počítač, který v roce 1997 porazil světového šachového mistra Garryho Kasparova. Používal kombinaci **brute-force vyhledávání**, **Alpha-Beta prořezávání** a **heuristických pravidel** trénovaných na hrách šachových velmistrů. Byl implementován na superpočítači s paralelním zpracováním a databází zahájení.
- **Význam:** Ukázal, že stroj může porazit člověka v komplexní strategické hře a otevřel cestu dalším AI v herních aplikacích.

Watson

- **Vývojce:** IBM
- **Účel:** Řešení otázek v přirozeném jazyce (NLP).
- **Popis:** Zvítězil v soutěži *Jeopardy!* v roce 2011 nad nejlepšími lidskými hráči. Využíval **zpracování přirozeného jazyka**, **analýzu velkých dat**, **strojové učení** a **vyhledávání informací**.
- **Význam:** Demonstroval schopnost AI pracovat s jazykem a znalostmi, otevřel cestu pro využití AI v oblastech jako medicína, právní poradenství nebo finance.

AlphaGo

- **Vývojce:** DeepMind (Google)
- **Účel:** Hraní deskové hry Go.
- **Popis:** V roce 2016 porazil světového mistra Lee Sedola. Kombinoval **Monte Carlo Tree Search**, **neuronové sítě** a **reinforcement learning**. AlphaGo Zero, jeho pozdější verze, se učil hrát zcela sám bez lidských dat.
- **Význam:** Překonal hranice tehdejší AI v oblasti her a ukázal schopnost učit se bez lidských vzorů.

Eliza

- **Vývojce:** Joseph Weizenbaum (1960s)

- **Účel:** Simulace psychoterapeuta.
- **Popis:** Jeden z prvních chatbotů, založený na **pattern matchingu** a klíčových slovech. Uměl reagovat generickými frázemi na vstupy typu „Jsem smutný“ apod.
- **Význam:** Ukázal, že lidé mohou přisuzovat strojům lidské vlastnosti (**Eliza efekt**). Nešlo o skutečnou AI, ale psychologický experiment.

Shakey

- **Vývojce:** SRI International (1966–1972)
- **Účel:** Autonomní robot s pohybem a plánováním.
- **Popis:** První robot, který kombinoval **vnímání prostředí**, **plánování** (jazyk STRIPS: P, A, I, G), a **vykonávání úloh**. Uměl se sám rozhodnout, jak splnit zadaný cíl (např. přesunout krabici). Implementován v jazyce Lisp, představil také první využití **A*** algoritmu.
- **Význam:** Zakladatel robotiky a plánování v umělé inteligenci.

DQN (Deep Q-Network)

- **Vývojce:** DeepMind (2013)
- **Účel:** Učení se hrát videohry bez znalosti pravidel.
- **Popis:** Kombinuje **Q-learning** s **hlubokými neuronovými sítěmi**. AI se učila hrát Atari hry (např. Pong, Breakout) pouze pozorováním obrazovky a získáváním odměn. Využívá **Stochastic Gradient Descent** k optimalizaci politiky.
- **Význam:** Ukázal, že AI může samostatně zvládnout úkoly jen na základě surových dat (pixely, odměny). Položil základ pro další metody v oblasti her i simulací.

2. Formální reprezentace problému AI

problém mnohorukého banditu, MDP, POMDP, hra v rozšířené formě

Formální modelování rozhodovacích úloh je základním stavebním kamenem inteligentních systémů. Každý typ problému umožňuje jinou míru pozorování a interakce s prostředím.

Problém mnohorukého banditu (Multi-Armed Bandit, MAB)

MAB modeluje situaci, kdy agent volí mezi akcemi (pažemi), z nichž každá přináší náhodnou odměnu s neznámým očekáváním. Cílem je maximalizovat kumulovaný výnos.

Formální definice:

- **Akce:** $A = \{a_1, \dots, a_k\}$
- **Odměny:** pro každé a_i náhodná proměnná $r_t^{(i)} \sim \nu_i$ s očekáváním $Q(a) = \mu_i$
- **Očekávaná hodnota akce:**

$$q_*(a) \doteq \mathbb{E}[R_t \mid A_t = a], \quad \forall a \in \{1, \dots, k\}$$

- **Cíl:**

$$\max_{\pi} \mathbb{E} \left[\sum_{t=1}^T r_t \right]$$

- Tato formalizace popisuje prostředí, kde agent volí z konečného množství akcí a pro každou dostává stochastickou odměnu, jejímž očekáváním se snaží řídit své rozhodování s cílem maximalizovat dlouhodobý zisk.

Algoritmy:

- **Greedy** – vždy vybírá akci s nejvyšším známým odhadem odměny:

$$a_t = \operatorname{argmax}_a Q(a)$$

- **Epsilon-Greedy** – s pravděpodobností $1 - \epsilon$ volí nejlepší známou akci, s ϵ náhodně:

$$a_t = \begin{cases} \operatorname{argmax}_a Q(a) & \text{s pravd. } 1 - \epsilon \\ \text{náhodná akce} & \text{s pravd. } \epsilon \end{cases}$$

- **UCB (Upper Confidence Bound)** – zohledňuje nejen očekávanou odměnu, ale i nejistotu v důsledku dosavadního počtu výběrů:

$$A_t \doteq \operatorname{arg max}_a \left[Q_t(a) + c \sqrt{\frac{\log t}{N_t(a)}} \right]$$

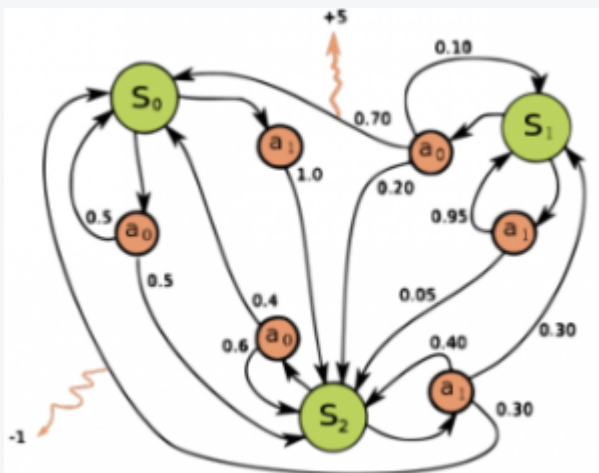
- kde:
 - $Q_t(a)$ je odhad očekávané odměny pro akci a ve čase t ,
 - $N_t(a)$ je počet výběrů akce a do času t ,
 - c je parametr určující míru explorační (čím větší c , tím více prozkoumáváme nové akce).
- **Nestacionární MAB** – modeluje časově proměnná očekávání $\mu_i(t)$. Používá se např. vážený klouzavý průměr nebo diskontování starších dat.

Poznámka:

- Problém odhaluje **exploration–exploitation dilemma**.
- Exploration = zkoumání nových akcí,
- Exploitation = výběr nejlepší doposud známé akce.
- Greedy přístup vede k **exploitingu**, epsilon-Greedy a UCB kombinují **exploring** a **exploiting**.

MDP (Markovský rozhodovací proces)

MDP je základní model sekvenčního rozhodování v prostředí, kde je aktuální stav plně pozorovatelný. Klíčovým rysem je **Markovova vlastnost** – budoucí stav závisí pouze na současném stavu a akci, nikoliv na celé minulosti.



Formální definice:

MDP je pětice:

$$(S, A, P, R, \gamma)$$

- kde:
 - S – konečná množina stavů,
 - A – konečná množina akcí,
 - $P(s'|s, a)$ – přechodová pravděpodobnost: pravděpodobnost přechodu do stavu s' po provedení akce a ve stavu s ,
 - $R(s, a)$ – očekávaná odměna za provedení akce a ve stavu s ,
 - $\gamma \in [0, 1)$ – diskontní faktor: určuje, jak silně jsou preferovány okamžité odměny před budoucími.

Cíl agenta:

Nalézt **politiku** $\pi : S \rightarrow A$, která maximalizuje očekávaný kumulovaný výnos:

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \mid s_0 = s \right]$$

Politika může být **deterministická** (každý stav má přesně jednu akci) nebo **stochastická** (každé akci ve stavu je přiřazena pravděpodobnost).

Hodnotové funkce:

- **Stavová hodnota** $V^\pi(s)$ – očekávaný výnos při dodržování politiky π ze stavu s .
- **Akční hodnota** $Q^\pi(s, a)$ – očekávaný výnos při provedení akce a ve stavu s a dalším následování politiky π .

Optimální politika π^* maximalizuje hodnotu ve všech stavech:

- $V^*(s) = \max_{\pi} V^\pi(s)$
- $Q^*(s, a) = \max_{\pi} Q^\pi(s, a)$

Bellmanovy rovnice:

Bellmanovy rovnice vyjadřují vztah mezi hodnotou daného stavu (nebo akce) a hodnotami následných stavů, čímž umožňují rekursivně dopočítat optimální strategii jako součet okamžité odměny a očekávané budoucí hodnoty.

Pro optimální hodnoty platí:

$$V^*(s) = \max_{a \in A} \left[R(s, a) + \gamma \sum_{s'} P(s'|s, a) V^*(s') \right]$$
$$Q^*(s, a) = R(s, a) + \gamma \sum_{s'} P(s'|s, a) \max_{a'} Q^*(s', a')$$

Algoritmy:

- **Value Iteration** – opakovaně aktualizuje $V(s)$ podle Bellmanovy rovnice, dokud nedojde ke konvergenci.
- **Policy Iteration** – střídavě provádí evaluaci politiky (V^π) a zlepšuje ji výběrem akcí s vyšší hodnotou.
- **Q-learning (off-policy)** – učí přímo $Q(s, a)$ ze vzorků přechodů bez znalosti P :

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

- **SARSA (on-policy)** – upravuje $Q(s, a)$ na základě akce vybrané politikou:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma Q(s', a') - Q(s, a)]$$

MDP poskytuje základní rámec pro plánování i učení z interakce, a jeho rozšíření vedla k hlubokému rozvoji oblasti reinforcement learningu.

POMDP (Partiálně pozorovatelný Markovský rozhodovací proces)

POMDP popisuje rozhodování v prostředí, kde stav není přímo pozorovatelný, ale agent dostává pouze nepřímé signály – pozorování. Musí si tedy udržovat **belief**, což je pravděpodobnostní rozdělení nad možnými stavy.

Formálně:

POMDP je sedmice:

$$(S, A, P, R, \Omega, O, \gamma)$$

- S – stavový prostor
- A – množina akcí
- $P(s'|s, a)$ – přechodová pravděpodobnost
- $R(s, a)$ – funkce odměny
- Ω – prostor pozorování

- $O(o|s', a)$ – pravděpodobnost pozorování o po přechodu do stavu s'
- $\gamma \in [0, 1)$ – diskontní faktor

Belief:

Belief b je distribuční odhad stavu systému. Po akci a a pozorování o se aktualizuje podle:

$$b'(s') = \eta \cdot O(o|s', a) \sum_{s \in S} P(s'|s, a) \cdot b(s)$$

kde η je normalizační konstanta zajišťující, že nový belief je opět pravděpodobnostní rozdělení.

Cíl agenta:

Nalézt **politiku** $\pi : \mathcal{B} \rightarrow \mathcal{A}$, která každému beliefu přiřadí akci. Cílem je maximalizovat očekávaný kumulovaný výnos:

$$V^\pi(b) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \mid b_0 = b \right]$$

kde očekávání se bere přes možné trajektorie stavů, pozorování a akcí, které vyplývají z původního beliefu a politiky.

Hodnotová funkce:

- **Belief value function** $V(b)$ – očekávaný výnos z beliefu b při dodržování politiky π
- Je to rozšíření hodnotové funkce z MDP, ale definované na prostoru pravděpodobnostních rozdělení místo diskrétních stavů.

POMDP model je velmi silný, ale výpočetně náročný, protože belief space je spojitý a vysokodimenzionální – řešení se často aproximuje pomocí metod jako **PBVI** nebo **HSVI**.

Algoritmy pro řešení POMDP:

Protože stav není přímo známý, POMDP se transformuje na MDP v prostoru beliefů.

- **Value Iteration over belief space**
 - Hodnotová funkce $V(b)$ je definována nad spojitým prostorem všech beliefů (pravděpodobnostních rozdělení).
 - U diskrétních POMDP je $V(b)$ piecewise-lineární a konvexní, ale výpočetně velmi náročná.
- **Point-based Value Iteration (PBVI)**
 - Aproximuje $V(b)$ jen na konečné množině reprezentativních beliefů.
 - Redukuje výpočetní náročnost a je vhodná pro větší stavové prostory.
- **Monte Carlo metody (např. POMCP)**
 - Používají vzorkování a simulace místo explicitního výpočtu.
 - **POMCP (Partially Observable Monte Carlo Planning)** kombinuje **MCTS** (Monte Carlo Tree Search) s postupnou aktualizací beliefu pomocí vzorků, bez nutnosti modelovat celý přechodový systém.
- **Policy Search**
 - Namísto hodnotové funkce přímo hledá politiku.
 - Obvykle používá parametrizované politiky (např. neuronové sítě) a **policy gradient** metody k optimalizaci očekávané odměny.

POMDP je obecně PSPACE-těžký problém, a proto se v praxi často používají přibližné metody. Přesto je klíčový v oblastech s částečnou observací, jako je robotika, zdravotnictví, plánování dialogu nebo navigace v neznámém prostředí.

Hra v rozšířené formě (Extensive-Form Games, EFGs)

Extensive-form games (EFGs) reprezentují vícekrokové rozhodovací situace pro více hráčů, často s neúplnou informací nebo náhodnými událostmi. Typicky jsou znázorňovány jako **stromy hry** (game trees).

- **Strom rozhodnutí** – každý uzel odpovídá rozhodnutí určitého hráče, hrany reprezentují možné akce.
 - Strom je obvykle **konečný** – hra má předem daný časový horizont (na rozdíl od MDP).
- **Informované množiny (Information sets)** – hráč si není vždy vědom, v jakém přesném uzlu stromu se nachází (např. kvůli nedokonalé informaci).
 - Skupina uzlů, mezi kterými nerozlišuje, tvoří jeho informační množinu.
- **Výplaty (utilities)** – definované pouze v **terminálních uzlech** stromu hry.
 - Každý hráč má přiřazen užitek v závislosti na dosaženém výsledku.
- **Dynamika** – každý stav má přiřazeného hráče (včetně přírody), který v něm rozhoduje o dalším kroku.
 - Hra tak popisuje střídání rozhodnutí různých aktérů (hráčů).

Hry v rozšířené formě se používají v oblasti víceagentních systémů, ekonomie, plánování a studia strategie. Vzhledem ke složitosti neúplné informace vyžadují často speciální algoritmy jako **regret minimization** nebo **Nashova rovnováha**.

3. Metody prohledávání stavového prostoru

DFS, BFS, ID-DFS, Dijkstra, A*

Prohledávání stavového prostoru je klíčovou technikou v umělé inteligenci, používanou např. při řešení hádanek, plánování, nebo hledání cest v grafech. Každá metoda má specifické chování, výhody a vhodné aplikace v závislosti na charakteru prostoru.

DFS (Depth-First Search – prohledávání do hloubky)

Prozkoumává nejprve co nejhlubší uzly, dokud nenarazí na cíl nebo slepou větev.

- **Algoritmus:**
 - Vlož počáteční stav na zásobník.
 - Opakuj:
 - Odeber vrchol ze zásobníku.
 - Pokud je cílový, skonči.
 - Jinak rozšiř následníky a vlož je na zásobník.
- **Vlastnosti:**
 - Paměť: $O(d)$ pro hloubku d
 - Nenajde nejkratší cestu
 - Riziko zacyklení (nutno pamatovat navštívené stavy)

BFS (Breadth-First Search – prohledávání do šířky)

Rozšiřuje stavy v pořadí jejich vzdálenosti od počátečního.

- **Algoritmus:**
 - Vlož počáteční stav do fronty.
 - Opakuj:
 - Odeber vrchol z fronty.
 - Pokud je cílový, skonči.
 - Jinak rozšiř následníky a vlož je do fronty.
- **Vlastnosti:**
 - Paměť: $O(b^d)$, kde b je branching factor a d hloubka řešení
 - Najde optimální cestu (pro jednotné náklady)
 - Náročné na paměť

ID-DFS (Iterative Deepening DFS)

Kombinuje výhody DFS a BFS – využívá malou paměť DFS, ale garantuje nalezení optimální cesty jako BFS.

- **Algoritmus:**
 - Pro $l = 1, 2, 3, \dots$:
 - Proveď DFS s omezením hloubky l

- **Vlastnosti:**
 - Paměť $O(d)$
 - Najde optimální řešení při jednotných nákladech
 - Časová rezie díky opakovanému průchodu, ale asymptoticky efektivní

Dijkstra

Hledá nejkratší cestu v grafu s nezápornými náklady hran.

- **Algoritmus:**
 1. Inicializuj vzdálenost všech uzlů na nekonečno, počáteční uzel na 0.
 2. Vlož všechny uzly do prioritní fronty.
 3. Opakuj:
 - I. Vyber uzel u s nejmenší vzdáleností.
 - II. Pro každého souseda v : pokud $\text{dist}[v] > \text{dist}[u] + \text{cost}(u, v)$, aktualizuj.
- **Vlastnosti:**
 1. Garantuje optimální řešení.
 2. Efektivní při známých a fixních nákladech.
 3. Funguje v obecné třídě grafů s nezápornými náklady.
- Dijkstra je speciální případ Uniform-Cost Search a může být použit k nalezení nejkratší cesty z jednoho bodu do všech ostatních.

A* (A-Star)

Rozšíření Dijkstry o heuristiku $h(n)$, která odhaduje náklady z uzlu n do cíle.

- **Algoritmus:**
 1. Inicializuj frontu uzlů s prioritou $f(n) = g(n) + h(n)$.
 2. Opakuj:
 - I. Odeber uzel s nejmenším $f(n)$.
 - II. Pokud je cílový, skonči.
 - III. Jinak rozšiř následníky a aktualizuj jejich g a f .
- **Heuristiky:**
 1. **Admisibilní (přípustná)** – nikdy nepřeceňuje skutečné náklady (tj. $h(n) \leq h^*(n)$), což zaručuje optimálnost.
 2. **Konzistentní (monotónní, consistent)** – platí $h(n) \leq c(n, n') + h(n')$, což zajišťuje, že se f hodnoty uzlů nemohou zmenšovat při cestě stromem.
 3. **Inadmisibilní** – rychlejší, ale nemusí najít optimální řešení.
- **Vlastnosti:**
 1. Při použití konzistentní heuristiky je A* optimální a rozšiřuje minimum uzlů mezi všemi optimálními algoritmy.
 2. A* expanduje všechny uzly s $f(n) < C^*$, kde C^* je náklad optimální cesty.
 3. Efektivita závisí na informativnosti heuristiky – čím blíže je $h(n)$ ke skutečné ceně, tím méně uzlů A* expanduje.
- **Využití:**
 1. Plánování pohybu (robotika, hry).
 2. Vyhledávání cest v grafech a mapách.
 3. Pokud $h(n) = 0$, chování jako Dijkstra.
- **Iterative Deepening A*:**
 1. Varianta A*, která omezuje průzkum na uzly s $f(n) \leq c$ a postupně zvyšuje c .
 2. Úspornější na paměť než klasické A*, vhodné pro velmi hluboké nebo rozsáhlé prostory.
 3. Další varianty:
 - Recursive Best-First Search (RBFS)

- Memory-bounded A* (MA*, SMA*)

Shrnutí heuristik

- **Heuristika** je funkce $h : S \rightarrow \mathbb{R}^+$, která odhaduje zbývající náklady do cíle.
- **Admisibilní heuristika** – nikdy nepřeceňuje náklady (optimální řešení).
- **Konzistentní heuristika** – splňuje $h(n) \leq c(n, n') + h(n')$, zajišťuje monotónnost $f(n)$.
- Každá konzistentní heuristika je zároveň admisibilní.
- Neinformativní heuristika (např. $h(n) = 0$) vede na Dijkstra.
- Příliš silná heuristika (např. $h(n) > h^*(n)$) může zrychlit výpočet, ale ztrácí optimálnost.

4. Algoritmy posilovaného učení

policy evaluation, policy improvement, policy iteration, value iteration, Q-learning

Základní pojmy

- **Politika (policy)** $\pi(a|s)$ Pravděpodobnostní pravidlo, které říká, jakou akci a agent zvolí ve stavu s . Cílem posilovaného učení je najít **optimální politiku** π^* , která maximalizuje dlouhodobou očekávanou odměnu.
- **Hodnotová funkce stavů** $V^\pi(s)$ Očekávaná diskontovaná kumulovaná odměna, kterou agent získá, pokud začne ve stavu s a dále se řídí politikou π .
- **Akční hodnotová funkce** $Q^\pi(s, a)$ Očekávaná diskontovaná odměna, pokud agent ve stavu s zvolí akci a a poté následuje politiku π .

Příklady reprezentace politik

- **Deterministická pravidlová** (reaktivní): *Pokud přede mnou není překážka, jdi rovně; jinak zatoč doprava.*
- **Tabulková stochastická**: explicitně uvedené pravděpodobnosti akcí v každém stavu.
- **Funkční aproximátor**: např. neuronová síť, která ze stavu přímo vrací distribuci akcí.

Všechny algoritmy níže pracují s prostředím modelovaným jako **Markov Decision Process (MDP)** definovaným čtveřicí $(\mathcal{S}, \mathcal{A}, P, R)$, kde $P(s'|s, a)$ je pravděpodobnost přechodu a $R(s, a, s')$ jednorázová odměna.

Policy Evaluation

Úkol. Spočítat $V^\pi(s)$ pro danou politiku π .

$$V^\pi(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a), [R(s, a, s') + \gamma V^\pi(s')],$$

kde $\gamma \in [0, 1)$ je diskontní faktor, jenž určuje, jak moc agent „přeje“ budoucím odměnám.

Co rovnice říká? Hodnota stavu je průměr (podle politiky a dynamiky prostředí) okamžité odměny R a hodnoty následujícího stavu, zlevněné faktorem γ .

Proč je to užitečné?

Jak se to počítá v praxi? Policy evaluation řešíme vždy jen pro *jednu* konkrétní politiku, nikoliv pro celou (často nekonečnou) množinu všech možných politik.

- **Malé & známé MDP**: rovnici lze vyřešit iterací nebo řešením soustavy lineárních rovnic.
- **Velké / neznámé MDP**: hodnotu V^π odhadujeme ze vzorků – Monte Carlo nebo TD(0), TD(λ). Znalost V^π říká, „kolik stojí“ být v každém stavu, pokud se budu chovat podle π . Je to klíčový stavební kámen pro zlepšování politik.

Příklad (Bludiště). Pro pevně danou politiku chození náhodně určíme, kolik kroků (-odměn) v průměru chybí do cíle z každého políčka.

Policy Improvement

Úkol. Z V^π vytvořit lepší politiku π' :

$$\pi'(s) = \arg \max_a \sum_{s'} P(s'|s, a), [R(s, a, s') + \gamma V^\pi(s')].$$

Co rovnice říká? V každém stavu vyber akci s největší očekávanou hodnotou budoucích odměn vzhledem k aktuálnímu odhadu V^π .

Proč je to užitečné? Zaručeně získáme politiku, která není horší než π (tzv. policy improvement theorem).

Příklad. Pokud v rohu bludiště vede akce *nahoru* k vyšší hodnotě než *doleva*, začneme v tomto stavu preferovat *nahoru*.

Jak policy improvement funguje v praxi

1. **Start:** vyber libovolnou počáteční politiku π_0 (třeba náhodnou).
2. **Policy evaluation:** odhadni hodnoty V^{π_0} .
3. **Greedy krok:** pro každý stav s spočítej očekávané výnosy jednotlivých akcí a zvol akci s nejvyšší hodnotou
 $\Rightarrow$ vznikne nová politika π_1 .
4. **Opakuj:** pokud se π_1 liší od π_0 , vrať se na krok 2 (nyní s π_1). Jakmile se politika přestane měnit, dospěli jsme k optimu.

Klíčové: **policy improvement je operátor** „vezmi existující politiku $\rightarrow$ vrať o kousek lepší“. Nehledá všechny politiky, jen lokálně zlepší tu, kterou dodáš.

Aproximované a stochastické politiky

Pokud je politika **parametrizovaná** (např. neuronová síť $\pi_\theta(a | s)$ nebo soft-max nad lineárním modelem), *greedy přepsání tabulky nahradíme gradientovým krokem*:

$$\theta \leftarrow \theta + \eta, \nabla_\theta J(\theta), \quad \text{kde } J(\theta) = \mathbb{E}_{s \sim d^\pi, a \sim \pi_\theta} [Q^\pi(s, a)].$$

- Gradient $\nabla_\theta J$ říká, **jak pohnout parametry, aby se zvýšila pravděpodobnost akcí s vyšším $Q^\pi(s, a)$.**
- V praxi se počítá stochasticky (REINFORCE, actor-critic, PPO...).
- **Intuice:** učíš síť, aby častěji nabízela akce, které aktuální odhad hodnot považuje za výhodné.

Policy Iteration

1. **Policy Evaluation:** spočti V^π
2. **Policy Improvement:** vytvoř π'
3. Opakuj s novou politikou, dokud se již nezmění.

Metoda konverguje k optimální politice π^* .

Použití. Vhodné pro MDP s plným modelem; rychle konverguje v malých stavových prostorech (např. diskretizované robotické úlohy).

Value Iteration

Value iteration kombinuje **policy evaluation** (odhad V) a **policy improvement** (výběr nejlepší akce) do *jediného* kroku:

$$V_{k+1}(s) = \max_a \sum_{s'} P(s'|s, a), [R(s, a, s') + \gamma V_k(s')].$$

Po konvergenci se politika získá:

$$\pi^*(s) = \arg \max_a \sum_{s'} P(s'|s, a), [R(s, a, s') + \gamma V(s')].$$

Vhodné pro velké MDP, kde je levnější aktualizovat pouze $V(s)$.

Q-learning

Q-learning je **model-free** off-policy algoritmus, který se učí optimální akční hodnoty $Q^*(s, a)$ přímo z interakcí s prostředím, aniž by potřeboval znát přechodové pravděpodobnosti $P(s'|s, a)$ nebo funkci odměn $R(s, a, s')$.

Aktualizační rovnice

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

Vysvětlení jednotlivých termů:

- $Q(s, a)$ - současný odhad hodnoty akce a ve stavu s
- $\alpha \in (0, 1]$ - **learning rate** (učicí rychlost): jak rychle zapomínáme staré odhady ve prospěch nových zkušeností
 - Malé α : pomalé učení, ale stabilnější konvergence
 - Velké α : rychlé učení, ale může oscilovat
- r - okamžitá odměna získaná po provedení akce a ve stavu s
- $\gamma \in [0, 1]$ - **discount factor**: jak moc vážíme budoucí odměny
 - $\gamma = 0$: agent se zajímá jen o okamžité odměny
 - $\gamma \rightarrow 1$: agent více váží dlouhodobé odměny
- $\max_{a'} Q(s', a')$ - odhad nejlepší možné hodnoty v následujícím stavu s'
- $r + \gamma \max_{a'} Q(s', a')$ - **TD target** (temporal difference target): náš nový odhad správné hodnoty $Q(s, a)$
- $r + \gamma \max_{a'} Q(s', a') - Q(s, a)$ - **TD error**: rozdíl mezi novým odhadem a současnou hodnotou

Intuice za rovnicí

Rovnice říká: "Aktualizuj svůj odhad hodnoty akce směrem k tomu, co jsi právě zažil (okamžitá odměna) plus nejlepší hodnota, kterou očekáváš v dalším stavu."

Praktický příklad: Q-tabulka pro Gridworld

Představme si jednoduché bludiště 3×3, kde agent začíná v levém horním rohu (0,0) a cílem je dostat se do pravého dolního rohu (2,2).

Prostředí:

S . .
. X . S = start, G = goal, X = překážka, . = volné pole
. . G

Možné akce: ↑ (nahoru), ↓ (dolů), ← (doleva), → (doprava)

Odměny: +10 za dosažení cíle, -1 za každý krok, -10 za náraz do zdi/překážky

Počáteční Q-tabulka (náhodně inicializovaná)

Stav	↑	↓	←	→
(0,0)	0.1	-0.3	0.2	0.4
(0,1)	0.0	0.1	-0.1	0.3
(0,2)	0.2	0.0	-0.2	0.1
(1,0)	-0.1	0.2	0.0	-0.3
(1,2)	0.1	0.3	0.0	-0.1
(2,0)	0.0	-0.1	0.2	0.4
(2,1)	-0.2	0.1	0.3	0.0

(2,2)	0.0	0.0	0.0	0.0
-------	-----	-----	-----	-----

Průběh učení - příklad kroku

Situace: Agent je ve stavu (0,0), zvolí akci $\rightarrow$ (doprava), dostane odměnu $r = -1$, dostane se do stavu (0,1).

Parametry: $\alpha = 0.1, \gamma = 0.9$

Aktualizace:

1. Současná hodnota: $Q(0, 0, \rightarrow) = 0.4$ 2. Odměna: $r = -1$
2. Nejlepší hodnota v novém stavu: $\max_{a'} Q(0, 1, a') = \max(0.0, 0.1, -0.1, 0.3) = 0.3$
3. TD target: $r + \gamma \max_{a'} Q(s', a') = -1 + 0.9 \times 0.3 = -0.73$
4. TD error: $-0.73 - 0.4 = -1.13$
5. Nová hodnota: $Q(0, 0, \rightarrow) = 0.4 + 0.1 \times (-1.13) = 0.287$

Q-tabulka po několika tisících epizodách

Stav	$\uparrow$	$\downarrow$	$\leftarrow$	$\rightarrow$
(0,0)	-8.2	5.4	-8.1	6.1
(0,1)	-7.1	6.8	-7.8	7.2
(0,2)	-6.3	7.9	-7.2	8.1
(1,0)	4.2	-6.1	-7.9	-8.5
(1,2)	6.8	8.9	-6.4	9.2
(2,0)	-5.4	-6.8	-7.1	7.8
(2,1)	-4.2	-5.1	8.4	9.7
(2,2)	0.0	0.0	0.0	0.0

Optimální politika (vyber akci s nejvyšší hodnotou): - (0,0): $\rightarrow$ (doprava) - (0,1): $\rightarrow$ (doprava) - (0,2): $\rightarrow$ (doprava) - (1,0): $\uparrow$ (nahoru) - (1,2): $\rightarrow$ (doprava) - (2,0): $\rightarrow$ (doprava) - (2,1): $\rightarrow$ (doprava)

Klíčové vlastnosti Q-learningu

1. **Model-free:** Nepotřebuje znát $P(s'|s, a)$ ani $R(s, a, s')$
2. **Off-policy:** Může se učit optimální politiku, i když jedná podle jiné (např. ϵ -greedy)
3. **Konvergence:** Za určitých podmínek (každý stav-akce navštíven nekonečněkrát, klesající learning rate) konverguje k Q^*
4. **Exploration vs Exploitation:** Často kombinováno s ϵ -greedy strategií pro vyvážení průzkumu a využívání

Praktické tipy

- **Inicializace:** Q-tabulku obvykle inicializujeme optimisticky (vyšší hodnoty podporují exploration)
- **Learning rate:** Často se snižuje v čase (např. $\alpha_t = \alpha_0 / (1 + t)$)
- **Exploration:** ϵ -greedy s postupně klesajícím ϵ je standardní volba

Příklad použití: Herní agent se z opakovaného hraní naučí strategii, aniž by znal pravidla hry předem. Q-learning úspěšně natrénoval agenty na Atari hry, šachy, nebo Go.

5. Algoritmy pro řešení her dvou hráčů

Formální model dvouhráčových her s dokonalou informací popisuje interakci dvou hráčů v diskrétní sekvenci tahů.

- **Hráči:** $P = \{1, 2\}$
- **Historie:** H je množina konečných posloupností tahů (stavů ve hře).
- **Akce:** $A(h)$ je množina akcí dostupných v bodě $h \in H$.
- **Terminální stavy:** $Z \subseteq H$ jsou stavy, kde hra končí.
- **Výplatní funkce:** $u : Z \rightarrow \mathbb{R}$ přiřazuje každému koncovému stavu číselné ohodnocení (užitek).

Hra je obvykle reprezentována stromem, ve kterém se hráči střídají v rozhodování a každý uzel odpovídá jednomu rozhodovacímu bodu.

minimax, alpha-beta prořezávání, negamax, niggascout, MCTS.

Minimax

Algoritmus pro rozhodování ve hrách s dokonalou informací (např. šachy, dáma), kde hráči střídavě maximalizují či minimalizují skóre.

Minimax je algoritmus pro rozhodování ve hrách s dokonalou informací, kde se hráči střídají a volí tahy s cílem maximalizovat (hráč 1) nebo minimalizovat (hráč 2) výsledné skóre.

- **Algoritmus:**
 - Prochází herní strom do dané hloubky nebo do koncových stavů (listů).
 - Maximalizující hráč vybírá tah s nejvyšším skóre, minimalizující s nejnižším.
 - Hodnoty se rekurzivně propagují zpět ke kořeni stromu.
- **Vlastnosti:**
 - Zaručuje optimální rozhodnutí, pokud je strom prohledán celý.
 - Exponenciální složitost $O(b^d)$, kde b je počet možných tahů a d hloubka.
- **Využití:**
 - Hry pro dva hráče s nulovým součtem – např. šachy, tic-tac-toe, reversi.

Alpha-Beta Pruning

Vylepšení Minimaxu, které ořezává větve stromu, jež nemohou ovlivnit konečné rozhodnutí – významně zrychluje hledání.

- **Algoritmus:**
 - Stejně jako Minimax, navíc udržuje dvě hodnoty:
 - α – nejlepší nalezený výsledek pro Max hráče (spodní mez).
 - β – nejlepší nalezený výsledek pro Min hráče (horní mez).
 - Pokud během prohledávání platí $\alpha \geq \beta$, větev se **ořízne** (dál se neprohledává).
- **Vlastnosti:**
 - Výsledky stejné jako Minimax, ale často prohledá podstatně méně uzlů (až $\sqrt{N}$ oproti N).
 - Maximální efektivita při dobrém pořadí tahů (nejlepší tahy prohledávány první).
- **Využití:**
 - Hry pro dva hráče, kde je důležité rychlé prohledání (šachy, go, reversi).

Negamax

Zjednodušení Minimaxu pro hry s nulovým součtem – místo dvou funkcí (max, min) používá jednu a neguje hodnoty.

- **Algoritmus:**
 - Reprezentuje úlohu tak, že vždy maximalizuje skóre pro "aktuálního hráče".

- Po každém kroku neguje návratovou hodnotu: $score = -negamax(s', depth - 1)$
- Tím stačí jedna rekurzivní funkce místo dvou (min, max).

- **Vlastnosti:**

- Výsledky stejné jako Minimax, jednodušší implementace.
- Snadné rozšíření o Alpha-Beta pruning.

- **Využití:**

- Hry s nulovým součtem, kde jsou hodnoty pro oba hráče přesně opačné.

Niggascout (Principal Variation Search, PVS)

Další vylepšení algoritmu Alpha-Beta (resp. Negamax), které využívá předpoklad, že akce jsou předem dobře seřazené (např. pomocí heuristiky). Tím výrazně urychluje prohledávání.

- **Algoritmus:**

- První tah prohledá s plným oknem $[\alpha, \beta]$.
- Ostatní tahy nejprve s "null window" $[\alpha, \alpha + 1]$ (rychlý test).
- Pokud výsledek překročí α , provede se "re-search" s plným oknem.
- Průběžně aktualizuje α a β , stejně jako Alpha-Beta.

- **Vlastnosti:**

- Ještě rychlejší než Alpha-Beta při dobrém pořadí tahů (většina tahů se nemusí detailně zkoumat).
- Výsledky stejné jako Alpha-Beta/Negamax.

- **Využití:**

- Šachové, reversi a jiné herní enginy, kde záleží na hlubokém a rychlém prohledávání.

MCTS (Monte Carlo Tree Search)

Heuristický algoritmus pro rozhodování v rozsáhlých nebo neúplných herních stromech, založený na náhodném simulování partií z aktuální pozice.

- **Algoritmus** (čtyři fáze):

- **Selection** – Prochází strom podle nejlepší známé hodnoty (UCT, Upper Confidence Bound) až k listu.
- **Expansion** – Pokud list není terminální, přidá nový poduzel.
- **Simulation (Playout)** – Z nového uzlu simuluje hru do konce náhodnými tahy.
- **Backpropagation** – Výsledek simulace propaguje zpět ke kořeni (aktualizuje statistiky uzlů).

- **Vlastnosti:**

- Nevyžaduje heuristiku, stačí simulace.
- Schopný vyrovnat se s obrovskými stromy, neúplnou informací a pravděpodobnostními hrami.
- S rostoucím počtem simulací se blíží optimálnímu rozhodnutí.

- **Využití:**

- Hry s obrovským stavovým prostorem (Go, některé deskovky, poker, plánování akcí v AI).

6. Strukturovaná reprezentace znalostí

CSP, Scheduling, Situation calculus, STRIPS

Strukturovaná reprezentace znalostí se zaměřuje na formální způsoby, jakými lze reprezentovat a manipulovat se znalostmi v počítačových systémech. Používají se různé modely a formalismy pro popis problémů, plánování a inferenci.

CSP - Problémy splnění omezení (Constraint Satisfaction Problems)

Definice

CSP (constraint satisfaction problems) jsou úlohy, ve kterých hledáme přiřazení hodnot proměnným tak, aby byla splněna všechna zadaná omezení.

Formálně je CSP definováno jako trojice **(X, D, C)**, kde:

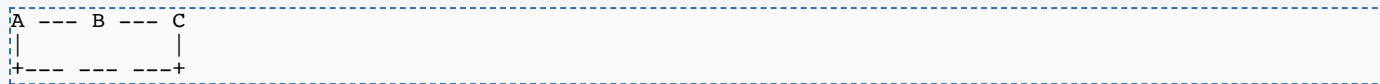
- **X** je množina proměnných: $\{X_1, X_2, \dots, X_n\}$
- **D** je množina domén: každá proměnná X_i má svou doménu D_i možných hodnot
- **C** je množina omezení: každé omezení specifikuje povolené kombinace hodnot pro podmnožinu proměnných

Cíl: Nalézt přiřazení hodnot proměnným tak, aby byla všechna omezení splněna.

Kompletní ukázka - Barvení grafu

Zadání problému

Máme graf se třemi uzly A, B, C, které jsou propojené takto:



Chceme obarvit každý uzel jednou ze tří barev tak, aby žádné dva sousední uzly neměly stejnou barvu.

Formální definice CSP

Proměnné: $X = \{A, B, C\}$

Domény:

- $D_A = \{\text{červená, zelená, modrá}\}$
- $D_B = \{\text{červená, zelená, modrá}\}$
- $D_C = \{\text{červená, zelená, modrá}\}$

Omezení:

- $C = \{A \neq B, B \neq C, A \neq C\}$
- A a B musí mít různé barvy (jsou sousední)
- B a C musí mít různé barvy (jsou sousední)
- A a C musí mít různé barvy (jsou sousední)

Řešení pomocí backtrackingu

Krok 1:

- Přiřadíme $A = \text{červená}$
- Současné přiřazení: $\{A: \text{červená}\}$
- Splněná omezení: žádná (zatím nemáme hodnoty pro B, C)

Krok 2:

- Zkusíme $B = \text{červená}$
- Současné přiřazení: $\{A: \text{červená}, B: \text{červená}\}$
- Kontrola omezení $A \neq B$: NESPLNĚNO (červená = červená) **Backtrack**
- zrušíme $B = \text{červená}$

Krok 3:

- Zkusíme $B = \text{zelená}$
- Současné přiřazení: $\{A: \text{červená}, B: \text{zelená}\}$
- Kontrola omezení $A \neq B$: SPLNĚNO (červená $\neq$ zelená)

Krok 4:

- Zkusíme $C = \text{červená}$
- Současné přiřazení: $\{A: \text{červená}, B: \text{zelená}, C: \text{červená}\}$
- Kontrola omezení:
 - $A \neq B$: SPLNĚNO (červená $\neq$ zelená)

- $B \neq C$: SPLNĚNO (zelená $\neq$ červená)
- $A \neq C$: NESPLNĚNO (červená = červená) **Backtrack**
- zrušíme $C = \text{červená}$

Krok 5:

- Zkusíme $C = \text{zelená}$
- Současné přiřazení: $\{A: \text{červená}, B: \text{zelená}, C: \text{zelená}\}$
- Kontrola omezení $B \neq C$: NESPLNĚNO (zelená = zelená) **Backtrack**
- zrušíme $C = \text{zelená}$

Krok 6:

- Zkusíme $C = \text{modrá}$
- Současné přiřazení: $\{A: \text{červená}, B: \text{zelená}, C: \text{modrá}\}$
- Kontrola všech omezení:
- $A \neq B$: SPLNĚNO (červená $\neq$ zelená)
- $B \neq C$: SPLNĚNO (zelená $\neq$ modrá)
- $A \neq C$: SPLNĚNO (červená $\neq$ modrá) - **ŘEŠENÍ NALEZENO!**

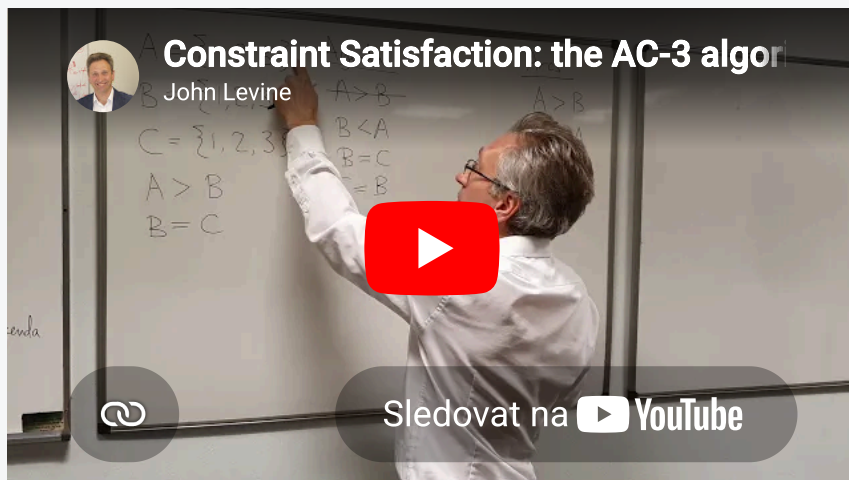
Nalezené řešení

- **A = červená**
- **B = zelená**
- **C = modrá**

Inference techniky

- **Forward checking:** Po každém přiřazení odstraní ze sousedních proměnných hodnoty, které by porušovaly omezení.
- **Arc consistency (AC):** Pro každou dvojici proměnných (X, Y) zajišťuje, že pro každou hodnotu X existuje nějaká hodnota Y , která neporušuje omezení.

AC-3 algoritmus využívá arc consistency, asi nejlepší vysvětlení je zde:



AC-3 algoritmus

Algoritmus pro dosažení lokální konzistence (arc consistency):

- Inicializuj frontu všech hran (omezení mezi proměnnými).
- Iterativně kontroluj každou hranu (X, Y) :
 - Pokud odstraníš nějakou hodnotu z domény X , znovu přidej všechny hrany směřující do X .
- Pokud některá doména zůstane prázdná $\rightarrow$ CSP nemá řešení.
- Nezaručuje nalezení řešení, jen redukuje prostor.

Heuristiky

- **Minimum Remaining Values (MRV):** Vyber proměnnou s nejmenším počtem možných hodnot – nejdřív se řeší „nejtěžší“ proměnné.
- **Least Constraining Value:** Vyber hodnotu, která ponechává nejvíce možností ostatním proměnným.
- **Backjumping, Dynamic Backtracking:** Chytřejší návraty při backtrackingu – vrací se více „na jistotu“.

Další příklady CSP

- **Sudoku:** Proměnné = políčka, domény = čísla 1-9, omezení = unikátnost v řádcích/sloupcích/čtvercích
- **N-queens:** Proměnné = pozice dam, domény = sloupce, omezení = žádné dva útočíci
- **Rozvrh hodin:** Proměnné = hodiny, domény = časy/místa, omezení = kolize učitelů/studentů

Scheduling

Plánování (scheduling) je speciální případ CSP, kde proměnné reprezentují **aktivitu** a domény představují **časové sloty** nebo **zdroje**, které lze k těmto aktivitám přiřadit.

Charakteristika

- Proměnné: jednotlivé aktivity nebo úlohy
- Domény: možné časy nebo dostupné zdroje
- Omezení:
 - **tvrdá omezení** (např. kapacita učeben, nepřekrývání aktivit, precedenční vztahy)
 - **měkká omezení** (např. preferované časy, minimalizace přechodů nebo náročnosti)

Ukázka – rozvrh:

- Aktivita: $X = \{\text{matematika, angličtina}\}$
- Domény: časové sloty $\{8 : 00, 9 : 00\}$
- Omezení: nelze mít matematiku a angličtinu zároveň $\rightarrow \text{matematika} \neq \text{angličtina}$

Situation Calculus

Situation Calculus je formální jazyk založený na predikátové logice, určený pro reprezentaci **měnících se světů**, zejména v kontextu **akcí a jejich efektů**. Často se používá pro plánování v AI.

Základní prvky

- **Situace** – abstraktní reprezentace historie provedených akcí (počáteční situace = S_0)
- **Akce** – operace, které mění stav světa (např. Move, PickUp)
- **Fluentní predikáty** – závisí na situaci, reprezentují stav světa (např. $\text{At}(\text{Robot}, \text{Room})$)
- **Rigidní predikáty** – nezávisí na situaci (např. $\text{Room}(\text{Room1})$)
- **Výsledková funkce:** $\text{do}(a, s)$ – situace, která vznikne provedením akce a ve stavu s

Standardní predikáty

- $\text{Holds}(f, s)$ – fluent f platí ve stavu s
- $\text{Poss}(a, s)$ – akce a je možná ve stavu s
- $\text{do}(a, s)$ – výsledek vykonání akce a ve stavu s

Ukázka:

```
Holds(At(Robot, Room1), S0)
Poss(Move(Robot, Room1, Room2), S)  $\Leftrightarrow$  Holds(At(Robot, Room1), S)
Holds(At(Robot, Room2), do(Move(Robot, Room1, Room2), S))
```

STRIPS

STRIPS (Stanford Research Institute Problem Solver) je formalismus pro reprezentaci plánovacích problémů ve formální logice. Byl původně vyvinut pro robota Shakey.

Formální definice

STRIPS problém je definován čtveřicí **(P, A, I, G)**, kde:

- **P** je množina všech atomických predikátů (např. $\text{At}(\text{Robot}, \text{Room1})$) – booleovské proměnné

- **A** je množina všech akcí, každá ve tvaru $\langle \text{pre}(a), \text{add}(a), \text{del}(a) \rangle$:
 - $\text{pre}(a)$ – předpoklady, které musí být splněny pro použití akce
 - $\text{add}(a)$ – predikáty, které po akci začnou platit
 - $\text{del}(a)$ – predikáty, které po akci přestanou platit
- **I** je počáteční stav – množina predikátů platných na začátku
- **G** je cílový stav – množina predikátů, které chceme dosáhnout

Každá akce má:

- **Preconditions:** podmínky pro použití akce
- **Add list:** co bude po akci pravdivé
- **Delete list:** co přestane platit

Ukázka – akce “přesuň robota z místnosti A do B”:

```
Action: Move(A, B)
Preconditions: At(Robot, A), Connected(A, B)
Add: At(Robot, B)
Delete: At(Robot, A)
```

Cíl plánování

Cílem je najít posloupnost akcí $a_1, a_2, \dots, a_k$, která při aplikaci na počáteční stav I postupně převede systém do stavu, kde $G \subseteq S_k$.

Řešení pomocí A^*

STRIPS problémy lze řešit algoritmem A^* , kde:

- **stav** je aktuální množina pravdivých predikátů
- **heuristika** může být například $h(s) = |G \setminus s|$ – počet cílů, které ještě nejsou splněny

Heuristiky a relaxace

- **Relaxace problému:** ignorujeme $\text{del}(a)$, tedy předpokládáme, že akce nic neodstraňují
 - Relaxovaná akce: $a^* = \langle \text{pre}(a), \text{add}(a), \emptyset \rangle$
- **Abstrakce:** např. odstranění některých predikátů nebo predikátové argumenty zjednodušíme

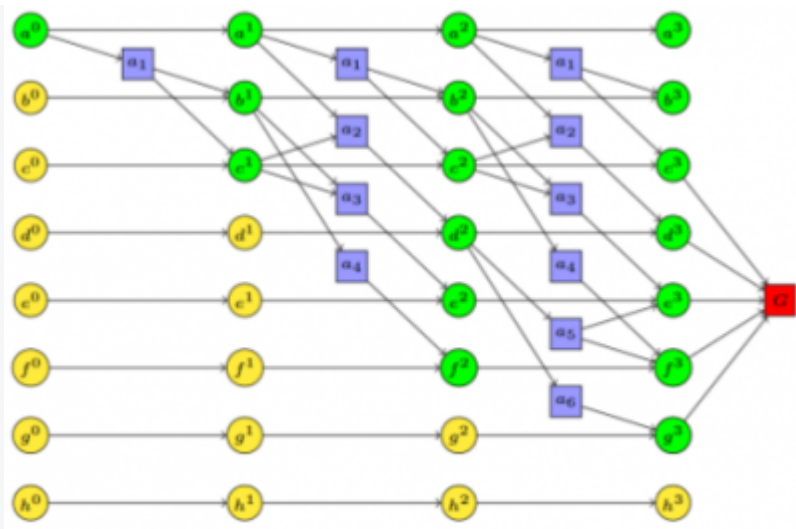
Plánovací graf (relaxovaný plánovací graf)

Graf dosažitelnosti:

- Střídají se úrovně predikátů P_i a akcí A_i
- $A_i = \{a \in A \mid \text{pre}(a) \subseteq P_i\}$
- $P_{i+1} = P_i \cup \{p \in \text{add}(a) \mid a \in A_i\}$
- Konstrukce končí, pokud $G \subseteq P_i$

Stejný graf se dá použít k výpočtu optimální heuristiky $h_{\max}$

- Vrchol, který náleží počátečnímu stavu se inicializuje na nulu
- Všechny vrcholy s akcemi jsou pak rovny maximu hodnot z předpokladů dané akce
- Do vrcholů se stavy (propozicemi) se pak zapíše nejlevnější akce, která do daného stavu vedla



7. Neurčitost v AI

maximalizace očekávané utility, Bayesovo pravidlo, Bayesovské sítě.

Neurčitost v umělé inteligenci se objevuje v situacích, kdy nemáme úplné nebo přesné informace o stavu světa. V takových případech je třeba rozhodovat na základě pravděpodobnostních modelů. Hlavními nástroji pro práci s neurčitostí jsou Bayesovo pravidlo, maximalizace očekávané utility a Bayesovské sítě.

Bayesovo pravidlo

Bayesovo pravidlo umožňuje aktualizovat pravděpodobnost hypotézy na základě nových dat. Jeho matematická formulace je:

$$P(H|D) = \frac{P(D|H) \cdot P(H)}{P(D)}$$

kde:

- H – hypotéza,
- D – pozorovaná data,
- $P(H|D)$ – posteriorní pravděpodobnost,
- $P(D|H)$ – pravděpodobnost dat za předpokladu hypotézy (likelihood),
- $P(H)$ – apriorní pravděpodobnost hypotézy,
- $P(D)$ – celková pravděpodobnost dat (normalizační konstanta).

Maximalizace očekávané utility

Racionální agent by měl volit takovou akci, která maximalizuje **očekávanou užitečnost**:

$$EU(a) = \sum_s P(s|a) \cdot U(s, a)$$

kde:

- a – akce,
- s – možný stav světa,
- $P(s|a)$ – pravděpodobnost stavu s po provedení akce a ,
- $U(s, a)$ – užitečnost výsledného stavu s při akci a .

Používá se v rozhodovacích sítích a obecně ve všech situacích, kde je třeba rozhodovat pod neurčitostí.

Bayesovské sítě

Bayesovské sítě jsou **orientované acyklické grafy (DAG)**, kde:

- uzly reprezentují náhodné proměnné,

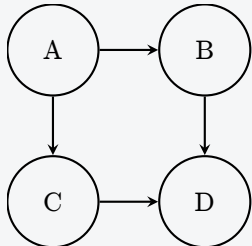
- hrany vyjadřují podmíněnou závislost (rodič ovlivňuje potomka),
- každá proměnná má tabulku podmíněných pravděpodobností (CPT).

Bayesovské sítě umožňují efektivní inferenci, tj. výpočet pravděpodobností nepozorovaných proměnných.

Celková distribuční pravděpodobnost v síti se rozpadá podle struktury grafu:

$$P(X_1, \dots, X_n) = \prod_{i=1}^n P(X_i \mid \text{rodiče}(X_i))$$

Příklad Bayesovské sítě



Tato Bayesovská síť ilustruje následující závislosti mezi náhodnými proměnnými:

- **A** je kořenová proměnná (nemá žádné předchůdce), která ovlivňuje:
 - **B**, tj. $P(B|A)$
 - **C**, tj. $P(C|A)$
- **D** je ovlivněna jak proměnnou **B**, tak i **C**, což znamená:
 - $P(D|B, C)$

Celková společná distribuční pravděpodobnost lze tedy rozepsat jako:

$$P(A, B, C, D) = P(A) \cdot P(B|A) \cdot P(C|A) \cdot P(D|B, C)$$

Naivní Bayesův klasifikátor

Naivní Bayesův klasifikátor je jednoduchý pravděpodobnostní klasifikátor založený na Bayesově teorému s **naivním** předpokladem **podmíněné nezávislosti** příznaků dané třídy.

Formální popis:

Nechť C je náhodná proměnná reprezentující třídu a $X = (X_1, X_2, \dots, X_n)$ jsou příznaky. Pomocí Bayesova pravidla:

$$P(C|X) = \frac{P(C) \cdot P(X|C)}{P(X)}$$

Za předpokladu nezávislosti:

$$P(X|C) = \prod_{i=1}^n P(X_i|C)$$

Klasifikace se provádí výběrem třídy c , která maximalizuje posteriorní pravděpodobnost:

$$\hat{c} = \arg \max_{c \in \mathcal{C}} P(C = c) \prod_{i=1}^n P(X_i|C = c)$$

Použití:

- detekce spamu,
- analýza sentimentu,
- lékařská diagnostika.

Skrytý Markovův model (Hidden Markov Model, HMM)

HMM je speciální typ Bayesovské sítě pro **časově závislé procesy**, kde:

- máme posloupnost **skrytých stavů** $Z_1, Z_2, \dots, Z_T$,
- z každého stavu je generována pozorovatelná veličina $X_1, X_2, \dots, X_T$,
- každý stav závisí pouze na předchozím: $P(Z_t|Z_{t-1})$,
- každé pozorování závisí pouze na současném stavu: $P(X_t|Z_t)$.

Společná pravděpodobnost:

$$P(Z_{1:T}, X_{1:T}) = P(Z_1) \cdot \prod_{t=2}^T P(Z_t|Z_{t-1}) \cdot \prod_{t=1}^T P(X_t|Z_t)$$

Algoritmy:

- **Forward-backward** – výpočet marginálních pravděpodobností,
- **Viterbi** – nalezení nejpravděpodobnější sekvence skrytých stavů,
- **Baum-Welch** – EM algoritmus pro trénink HMM.

Použití:

- rozpoznávání řeči,
- strojový překlad,
- analýza časových řad,
- sledování objektů.

8. Řešení POMDP

PBVI, HSVI

Řešení POMDP

Řešení částečně pozorovatelných Markovových rozhodovacích procesů (POMDP) spočívá v nalezení optimální politiky na základě neúplné informace o stavu systému. V POMDP není stav přímo pozorovatelný – rozhodování se děje na základě tzv. **belief state** – pravděpodobnostního rozdělení přes možné stavy.

Formální definice

POMDP je definován jako sedmice $\langle \mathbf{S}, \mathbf{A}, \mathbf{T}, \mathbf{R}, \mathbf{\Omega}, \mathbf{O}, \gamma \rangle$, kde:

- $\mathbf{S}$ – množina stavů
- $\mathbf{A}$ – množina akcí
- $\mathbf{T}(\mathbf{s}'|\mathbf{s}, \mathbf{a})$ – přechodová pravděpodobnost
- $\mathbf{R}(\mathbf{s}, \mathbf{a})$ – odměnová funkce
- $\mathbf{\Omega}$ – množina pozorování
- $\mathbf{O}(\mathbf{o}|\mathbf{s}', \mathbf{a})$ – pravděpodobnost pozorování $\mathbf{o}$ po akci $\mathbf{a}$ a přechodu do stavu $\mathbf{s}'$
- γ – diskontní faktor

Hodnotová funkce je definovaná nad belief space a je konvexní. Přesné řešení je výpočetně náročné, proto používáme aproximační algoritmy.

Řešení problémů částečně pozorovatelných Markovových rozhodovacích procesů (POMDP) spočívá v nalezení optimální politiky, která maximalizuje očekávaný výnos na základě neúplné informace o stavu systému. V POMDP je totiž skutečný stav systému skrytý (latentní) a rozhodnutí je činěno na základě tzv. věrohodnostního rozdělení (belief state) – pravděpodobnostního rozdělení přes možné stavy daného systému.

Hodnotová funkce pro POMDP je definována nad věrohodnostními stavy a je konvexní. Přesné řešení POMDP je výpočetně neproveditelné pro větší problémy, proto se používají aproximační metody. Mezi ně patří:

Point-Based Value Iteration (PBVI)

PBVI aproximuje hodnotovou funkci pouze v konečné množině **belief pointů** $B = \{b_1, b_2, \dots, b_n\}$. Pro tyto body iterativně provádí Bellmanovy aktualizace:

1. Inicializace hodnotové funkce V_0 . 2. Iterace pro každý $b \in B$:

$$V_{i+1}(b) = \max_{a \in A} \left[R(b, a) + \gamma \sum_{o \in \Omega} P(o|b, a) \cdot V_i(b_{a,o}) \right]$$

- $b_{a,o}$ je nová víra (belief) po akci a a pozorování o
- Hodnotová funkce se reprezentuje pomocí **α -vektorů**, které aproximují výnos pro jednotlivé akce a stavy

Tímto způsobem PBVI konstruuje aproximaci optimality pomocí hodnotové funkce složené z tzv. α -vektorů.

PBVI výrazně zjednodušuje výpočet tím, že nepočítá hodnotu v celém belief prostoru.

Heuristic Search Value Iteration (HSVI)

HSVI kombinuje heuristické řízení a aproximaci hodnotové funkce pomocí horní a dolní meze:

1. **Inicializace:**

- Dolní mez V^- : jednoduché politiky (např. fixní akce)
- Horní mez V^+ : řešení odpovídající plně pozorovatelnému MDP

2. **Heuristické prohledávání:**

- Vybíráme beliefy, kde je největší rozdíl mezi V^+ a V^-
- Průchod stromem víry (belief tree) řízený rozdílem mezi odhady

3. **Update:**

- Dolní mez – přidání nových α -vektorů
- Horní mez – konvexní obálka a přepočítání odhadu

Algoritmus HSVI: 1. Udržuje dolní odhad V^- a horní odhad V^+ hodnotové funkce. 2. Používá hloubkové prohledávání stromu víry řízené heuristikou k výběru větví, kde je největší rozdíl mezi V^+ a V^- . 3. Provádí zálohování hodnot a zpřesňuje odhady na cestě k listům stromu.

HSVI garantuje ϵ -optimalitu: algoritmus konverguje, když rozdíl mezi V^+ a V^- je menší než dané ϵ .

HSVI zachovává horní a dolní meze hodnotové funkce a iterativně je zpřesňuje pouze v místech, která ovlivňují politiku.

HSVI kombinuje výpočetní efektivitu s garancemi na kvalitu aproximace: algoritmus konverguje k ϵ -optimální politice, pokud se rozdíl mezi horním a dolním odhadem zmenší pod ϵ .

Oba algoritmy se osvědčily pro řešení středně velkých POMDP problémů, kde přesné metody selhávají kvůli výpočetní neúnosnosti.

Trace: • [b4b36zui](#)